**Национальный исследовательский университет "Высшая школа экономики"**
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

| СОГЛАСОВАНО | УТВЕРЖДЕНО |
|---|---|
| Старший преподаватель: Департамент анализа данных и искусственного интеллекта, НИУ ВШЭ | Академический руководитель образовательной программы "Программная инженерия", старший преподаватель департамента программной инженерии |
| _____ А. А. Паринов | _____ Н. А. Павлочев |
| «___» _____ 2026 г. | «___» _____ 2026 г. |

# ЦИФРОВОЙ АССИСТЕНТ СТУДЕНТА: СЕРВЕРНАЯ ЧАСТЬ

## Программа и методика испытаний

## ЛИСТ УТВЕРЖДЕНИЯ

## RU.17701729.12.17 51 01-1-ЛУ

Исполнители:
Студент группы БПИ236
_____ / С. Е. Растворов /
«___» _____ 2026 г.

**2026**

| Подп. и дата | |
|---|---|
| Инв.№ дубл. | |
| Взам. инв.№ | |
| Подп. и дата | |
| Инв.№ подп | |

УТВЕРЖДЕН

RU.17701729.12.17 51 01-1-ЛУ

# ЦИФРОВОЙ АССИСТЕНТ СТУДЕНТА: СЕРВЕРНАЯ ЧАСТЬ

## Программа и методика испытаний

## RU.17701729.12.17 51 01-1

## Листов 20

**2026**

| Инв.№ подп | Подп. и дата | Взам. инв.№ | Инв.№ дубл. | Подп. и дата |
|---|---|---|---|---|
| | | | | |

## АННОТАЦИЯ

Программа и методика испытаний определяет объект, цель, средства, порядок и методы испытаний программного продукта «Цифровой Ассистент Студента: серверная часть».

Документ предназначен для проверки соответствия программы требованиям технического задания и фиксации критериев приемки результата разработки.

## СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ . . . 4
1.1. Наименование программы . . . 4
1.2. Краткая характеристика области применения программы . . . 4
2. ЦЕЛЬ ИСПЫТАНИЙ . . . 5
3. ТРЕБОВАНИЯ К ПРОГРАММЕ . . . 6
3.1. Требования к функциональным характеристикам . . . 6
3.1.1. Состав выполняемых функций . . . 6
3.2. Требования к входным и выходным данным . . . 7
3.2.1. Организация входных данных . . . 7
3.2.2. Организация выходных данных . . . 8
3.3. Требования к интерфейсу . . . 8
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ . . . 10
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ . . . 11
5.1. Технические средства . . . 11
5.2. Программные средства . . . 11
5.3. Порядок проведения испытаний . . . 11
5.4. Требования к персоналу . . . 12
6. МЕТОДЫ ИСПЫТАНИЙ . . . 14
6.1. Проверка требований к технической документации . . . 14
6.2. Проверка требований к интерфейсу . . . 14
6.3. Проверка требований к функциональным характеристикам . . . 14
6.3.1. Основной сценарий . . . 15
6.3.2. Подтверждение email и доступ к учетной записи . . . 15
6.3.3. Импорт и обновление каталога проектов . . . 15
6.3.4. Поиск и рекомендации . . . 15
6.3.5. Подача и обработка заявок . . . 16
6.3.6. Создание и модерация проекта . . . 16
6.3.7. Экспорт и административные операции . . . 16
6.3.8. Доменные события и синхронизация факультета . . . 16
6.3.9. Проверка API по группам маршрутов . . . 17
6.3.10. Проверка CI/CD и релизного контура . . . 17
6.3.11. Проверка наблюдаемости . . . 17
6.3.12. Проверка резервного копирования и восстановления . . . 18
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ . . . 19

# 1. ОБЪЕКТ ИСПЫТАНИЙ

## 1.1. Наименование программы

Наименование программы: «Цифровой Ассистент Студента: серверная часть».

Наименование программы на английском языке: «Digital Student Assistant».

## 1.2. Краткая характеристика области применения программы

Веб-платформа для подбора и сопровождения студенческих проектов ВШЭ. Система объединяет каталог проектов, импорт ЭПП из XLSX, роли студентов, заказчиков и сотрудников ЦППРП, модерацию, заявки, избранное, интересы и рекомендации. Основной контур реализован как Django + DRF с HTML-интерфейсом на Django templates, отдельным FastAPI ML-сервисом и graph-сервисом на Neo4j. Серверная часть публикует доменные события через outbox API доставки событий, а внешние потребители используют их для обновления индекса рекомендаций и графовой проекции. В актуальной версии добавлены подтверждение адреса электронной почты при регистрации, django-unfold admin, домен faculty для синхронизации преподавателей/публикаций/курсов и сопоставления научных руководителей проектов с профилями HSE. Целевое окружение использует PostgreSQL, Docker Compose, Nginx и сценарии для развертывания, резервного копирования и синхронизации окружения.

## 2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка корректности выполнения программой функций, изложенных в п. 4 «Требования к программе» документа «Техническое задание» из комплекта документации в соответствии с ЕСПД (Единой системой программной документации).

Для проекта «Цифровой Ассистент Студента: серверная часть» испытания направлены на подтверждение того, что серверная часть и веб-интерфейс корректно выполняют состав функций, установленный техническим заданием: регистрацию и подтверждение адреса электронной почты, получение токенов доступа после успешной проверки адреса, разграничение доступа по ролям, просмотр и обработку каталога проектов, работу с карточками проектов, подачу и отзыв заявок, модерацию, управление избранным и интересами, выдачу рекомендаций, импорт ЭПП из XLSX, обработку доменных событий через outbox, взаимодействие с сервисами рекомендаций и графа, а также функции административного и кабинетного доступа. Испытания также проверяют, что при выполнении указанных сценариев соблюдаются требования к валидации входных данных, кодам ответов API, сохранности и целостности данных, журналированию критичных действий и устойчивости обработки ошибок. Границы приемки определяются техническим заданием, программной документацией и контрактами API; выявленные отклонения фиксируются как замечания к приемке.

# 3. ТРЕБОВАНИЯ К ПРОГРАММЕ

## 3.1. Требования к функциональным характеристикам

### 3.1.1. Состав выполняемых функций

Программа должна соответствовать следующим функциональным требованиям, указанным в документе «Цифровой Ассистент Студента: серверная часть. Техническое задание», и проверяться по ним в ходе испытаний.

В рамках испытаний подлежат проверке функции серверной части и веб-интерфейса, обеспечивающие работу с каталогом проектов, заявками, модерацией, избранным, интересами, рекомендациями, импортом данных и административными операциями.

Для студента должны проверяться следующие функции:

1. Аутентификация и доступ к персональному разделу на основе учетной записи пользователя с подтверждением адреса электронной почты перед выдачей доступа к защищенным операциям.
2. Просмотр каталога проектов и карточек проектов.
3. Фильтрация и сортировка каталога по параметрам, предусмотренным реализованным интерфейсом и API.
4. Просмотр подробной информации о проекте, включая сведения, используемые при выборе и подаче заявки.
5. Управление избранными проектами.
6. Управление личными интересами и получение рекомендаций по интересам.
7. Подача заявки на проект, отзыв заявки и просмотр статуса заявки.

Для преподавателя или заказчика должны проверяться следующие функции:

1. Создание карточки проекта.
2. Редактирование карточки проекта.
3. Передача проекта на модерацию.
4. Просмотр списка заявок студентов на свои проекты.
5. Принятие или отклонение заявки.
6. Указание причины отклонения при выполнении соответствующей операции.
7. Управление квотами и параметрами набора, связанными с публикацией и обработкой проектов.

Для сотрудника ЦППРП или модератора должны проверяться следующие функции:

1. Просмотр очереди проектов, находящихся на модерации.
2. Возврат проекта на доработку с комментарием.
3. Публикация проекта и снятие проекта с публикации.
4. Просмотр и обработка заявок, связанных с модерацией и утверждением участников.
5. Просмотр агрегированных показателей по проектам, заявкам и утвержденным участникам.
6. Просмотр уведомлений, связанных с дедлайнами и состоянием обработки данных.

Для серверной части должны проверяться следующие функции:

1. Обработка API-запросов к каталогам, карточкам проектов, заявкам, избранному, интересам, рекомендациям, импорту и административным операциям.
2. Импорт проектов из XLSX-файлов и структурированных таблиц.
3. Очистка и нормализация данных при импорте.
4. Обновление данных каталога и связанных сущностей после изменения проектов и заявок.
5. Формирование ответов для клиентской части в формате JSON.
6. Формирование результатов семантического поиска и рекомендаций через ML-сервис с возможностью запасного эвристического режима.
7. Поддержка outbox-доставки событий для внешних потребителей.
8. Синхронизация домена faculty, включая преподавателей, публикации, курсы и сопоставление научных руководителей с проектами.

## 3.2. Требования к входным и выходным данным

### 3.2.1. Организация входных данных

Входными данными программы являются данные, вводимые пользователем через веб-интерфейс, а также структурированные данные, поступающие от внешних и внутренних сервисов в рамках работы серверной части.

Для клиентской части входными данными являются:

1. Учетные данные пользователя при входе в систему.
2. Код подтверждения электронной почты, используемый при завершении регистрации.
3. Параметры поиска, фильтрации и сортировки каталога проектов.
4. Данные формы создания и редактирования проекта.
5. Данные заявки на проект.
6. Данные для добавления или удаления проекта из избранного.
7. Данные личных интересов пользователя.
8. Данные для модерации, включая комментарии, причины отклонения и служебные действия по статусам проекта.
9. Данные для просмотра и редактирования профиля.

Для серверной части входными данными являются:

1. HTTP-запросы к REST API с данными в формате JSON.
2. XLSX-файлы, загружаемые для импорта проектов.
3. Структурированные данные из внутренних таблиц и доменных моделей, используемые при формировании каталога, заявок, рекомендаций и административных представлений.
4. События домена, поступающие в outbox-доставку и используемые внешними сервисами для актуализации поискового индекса, графовой проекции и связанных представлений.
5. Данные из факультетного домена, используемые для хранения сведений о преподавателях, публикациях, курсах и сопоставлении руководителей.

При проверке входных данных должны контролироваться:

1. Наличие обязательных полей.
2. Корректность типов данных.

3. Соответствие значений допустимым статусам, ролям и переходам состояния.
4. Корректность размера и формата загружаемых файлов.
5. Соответствие прав доступа роли пользователя.
6. Корректность ссылочной и доменной целостности при изменении объектов.

#### 3.2.2. Организация выходных данных

Выходные данные программы представляются пользователю через веб-интерфейс и через ответы API серверной части.

Для клиентской части выходными данными являются:

1. Страницы каталога проектов и карточек проектов.
2. Страницы личного кабинета, избранного, интересов и заявок.
3. Формы создания, редактирования и модерации проектов.
4. Формы просмотра и обработки заявок.
5. Сообщения об успешном выполнении операций и ошибках ввода.
6. Текстовые результаты рекомендаций и пояснения к ним, если они возвращаются интерфейсом.
7. Данные о подтверждении действий пользователя.

Для серверной части выходными данными являются:

1. Ответы API в формате JSON.
2. Коды HTTP-ответов, отражающие результат обработки запроса.
3. Детализированные сообщения об ошибках валидации, доступа и обработки.
4. Результаты поиска и рекомендаций, возвращаемые клиенту или внешним сервисам.
5. Экспортные файлы XLSX или CSV, формируемые для административных процедур и выгрузок.
6. Структурированные журналы и служебные данные для мониторинга и аудита.
7. События, публикуемые для последующей обработки внешними сервисами.

При проверке выходных данных должны контролироваться:

1. Корректность структуры ответа.
2. Соответствие формата данных ожидаемому типу потребителя.
3. Соответствие кодов ошибок характеру нарушения.
4. Наличие данных, необходимых для отображения состояния объектов и завершения пользовательских сценариев.
5. Корректность формирования экспортов и служебных сообщений.

### 3.3. Требования к интерфейсу

Клиентская часть системы должна предоставлять веб-интерфейс, обеспечивающий доступ к функциям программы через современный браузер без установки дополнительных расширений.

Общие требования к интерфейсу:

1. Интерфейс должен быть реализован в виде веб-приложения.
2. Элементы управления должны быть логически сгруппированы по выполняемым действиям.
3. Навигация по разделам должна быть единообразной.

4. Доступ к разделам и операциям должен ограничиваться ролью пользователя.
5. Ошибки ввода данных должны сопровождаться понятными сообщениями.
6. Для подтверждения значимых действий могут использоваться диалоговые окна.
7. Интерфейс должен корректно отображать состояние каталогов, заявок, модерации и профиля пользователя.

Интерфейс студента должен обеспечивать:

1. Просмотр каталога проектов в виде карточек.
2. Поиск, фильтрацию и сортировку проектов.
3. Просмотр подробной информации о проекте.
4. Подачу, отзыв и отслеживание статуса заявки.
5. Управление избранным и личными интересами.
6. Просмотр рекомендаций по интересам.

Интерфейс преподавателя или заказчика должен обеспечивать:

1. Создание и редактирование карточек проектов.
2. Передачу проектов на модерацию.
3. Просмотр заявок студентов.
4. Принятие или отклонение заявок.
5. Управление квотами и параметрами набора.

Интерфейс администратора или сотрудника ЦППРП должен обеспечивать:

1. Просмотр проектов, находящихся на модерации.
2. Принятие решений по публикации и возврату проектов на доработку.
3. Управление параметрами проектов и сроками их выполнения.
4. Просмотр агрегированных показателей и служебных уведомлений.

Для оценки интерфейса в ходе испытаний должны использоваться сценарии, соответствующие основным ролям системы: студент, заказчик и сотрудник ЦППРП.

## 4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытание должна быть представлена документация в следующем составе:

1. «Цифровой Ассистент Студента: серверная часть». Техническое задание (ГОСТ 19.201-78 [7]).
2. «Цифровой Ассистент Студента: серверная часть». Пояснительная записка (ГОСТ 19.404-79 [10]).
3. «Цифровой Ассистент Студента: серверная часть». Программа и методика испытаний (ГОСТ 19.301-79 [8]).
4. «Цифровой Ассистент Студента: серверная часть». Текст программы (ГОСТ 19.401-78 [9]).
5. «Цифровой Ассистент Студента: серверная часть». Руководство оператора (ГОСТ 19.505-79 [11]).

# 5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

## 5.1. Технические средства

Испытания программы проводятся на рабочей станции, способной развернуть контейнерный стенд проекта и открыть веб-интерфейс в современном браузере. Для проверки клиентской части достаточно компьютера с доступом в сеть и одним из поддерживаемых браузеров: Chrome, Firefox, Safari или Edge.

Для проверки серверной части и интеграционных сценариев используется локальный или тестовый стенд, собираемый через Docker Compose. В состав такого стенда входят серверная часть на Django и DRF, HTML-интерфейс на Django templates, сервис рекомендаций на FastAPI, сервис графовой проекции, PostgreSQL, Nginx и связанные контейнеры проекта. При проверке сценариев импорта и outbox-доставки также требуется доступ к файловой системе стенда и журналам контейнеров.

Отдельно для проверки графовой части требуется развернутый экземпляр Neo4j, поскольку тестирование обновления графовой проекции и связанных событий выполняется только при доступной графовой базе. Для сценариев подтверждения электронной почты требуется настроенный почтовый механизм окружения, используемый проектом в выбранной конфигурации.

## 5.2. Программные средства

Для подготовки и запуска испытаний применяются средства, подтвержденные техническим заданием, кодом и служебными документами проекта. Базовый набор включает Python 3.13+, Django 5+, FastAPI, PostgreSQL, Neo4j, Docker и Docker Compose. Для работы с зависимостями и командными сценариями используются `uv` и репозиторные скрипты проверки.

Автоматизированная проверка выполняется средствами репозитория: `./scripts/run-release-gate.sh`, `./scripts/uv-linters.sh`, `./scripts/uv-typecheck.sh`. Эти команды используются для контроля согласованности контракта, линтинга, статической проверки типов и готовности сборки к выпуску. Для валидации контейнерного окружения применяются конфигурации Docker Compose из каталога `infra`.

Для ручной проверки пользовательских сценариев применяются современные веб-браузеры, а для анализа результатов испытаний используются журналы приложения, структурированные JSON-логи, ответы API и страницы веб-интерфейса. При проверке роли администратора дополнительно используется административный интерфейс проекта, включая `django-unfold`.

## 5.3. Порядок проведения испытаний

Испытания выполняются последовательно, чтобы сначала подтвердить корректность развертывания, а затем проверить пользовательские и интеграционные сценарии.

1. Подготавливают окружение: устанавливают зависимости командой `uv sync --all-packages --group dev`, задают переменные окружения и собирают контейнерный стенд через Docker Compose.
2. Запускают служебные проверки проекта и убеждаются, что репозиторные команды контроля качества завершаются без ошибок.

3. Проверяют доступность базовых точек входа системы: `/health/`, `/ready/`, `/api/`, `/api/docs/`, `/api/v1/`, а также readiness ML- и graph-сервисов, если они входят в состав проверяемого стенда.
4. Проверяют сценарии аутентификации и подтверждения электронной почты, затем выполняют выпуск токена через `POST /api/v1/auth/token/` и убеждаются, что выдача токена недоступна неподтвержденному пользователю.
5. После получения токена последовательно проверяют основные группы маршрутов API: `account`, `users/me`, `projects`, `applications`, `recs`, `imports`, `faculty`, `outbox`, а также административные export- и moderation-ручки, доступные по роли пользователя.
6. Проверяют сценарии студента: просмотр каталога, карточку проекта, избранное, интересы, подачу и отзыв заявки, отображение статуса заявки и уведомлений.
7. Проверяют сценарии заказчика: создание и редактирование проекта, отправку на модерацию, просмотр заявок, принятие или отклонение заявок, управление квотами и окнами набора.
8. Проверяют сценарии ЦППРП: очередь модерации, возврат на доработку, публикацию или отклонение проекта, управление дедлайнами, шаблонами и просмотр агрегированных показателей.
9. Проверяют импорт проектов из XLSX, обновление данных через outbox и актуализацию рекомендаций. При доступном Neo4j дополнительно проверяют передачу событий в графовую проекцию и корректность ее обновления.
10. Для машинного взаимодействия отдельно проверяют чтение `outbox/events`, подтверждение `outbox/events/ack/` и контрольные точки потребителей `outbox/consumers/{consumer}/checkpoint/` с использованием служебного токена.
11. Для staging- и production-контура выполняют проверку после запуска: доступность `Nginx`, успешный ответ `web`, работоспособность Compose-конфигурации и корректность обслуживания запросов через прокси. Для внешней проверки используются адреса `https://dsa-staging.s3rg.ru/` и `https://dsa.s3rg.ru/`, а также их прикладные точки `/api/v1/`, `/api/docs/`, `/health/` или иные контрольные маршруты, предусмотренные текущей конфигурацией.
12. Проверяют observability-контур: наличие целей в `Prometheus`, доступность `Grafana`, загрузку подготовленной панели мониторинга и появление метрик по сервисам.
13. Проверяют эксплуатационные процедуры защиты данных: создание резервной копии, наличие манифеста и контрольной суммы, а также тестовое восстановление из `backup.sql` или `backup.sql.gz` на стенде.
14. Фиксируют результаты по каждому сценарию и сопоставляют их с требованиями технического задания, контрактами API и эксплуатационными регламентами проекта.

### 5.4. Требования к персоналу

Испытания должен выполнять специалист, который умеет работать с веб-приложением, запускать контейнерный стенд и пользоваться командной строкой для запуска репозиторных прове-

рок. Для проверки серверной части необходимы базовые навыки работы с Docker Compose, чтения журналов и анализа HTTP-ответов.

Для приемочных сценариев требуется участие пользователя соответствующей роли: студента, заказчика и представителя ЦППРП. Эти участники должны уметь выполнять типовые действия в веб-интерфейсе, но специальная техническая подготовка от них не требуется.

При проверке административных функций дополнительно требуется пользователь с правами модератора или администратора. Для сценариев импорта, outbox-доставки и графовой проекции желательно участие разработчика или тестировщика, знакомого со структурой проекта и составом его сервисов.

# 6. МЕТОДЫ ИСПЫТАНИЙ

## 6.1. Проверка требований к технической документации

Проверка требований к технической документации выполняется путем сверки состава и содержания представленных документов с перечнем, установленным техническим заданием и ГОСТ 19.301-79. В ходе испытаний проверяется наличие технического задания, пояснительной записки, программы и методики испытаний, текста программы и руководства оператора, а также их принадлежность к одной и той же программе "Цифровой Ассистент Студента: серверная часть". Дополнительно контролируется непротиворечивость терминов, обозначений ролей, наименований подсистем и состава функций, указанных в документах.

Исходные данные: комплект программной документации, подготовленный для сдачи и приемки, включая электронные версии документов. Действия: последовательно открыть каждый документ, проверить наличие обязательных разделов, сравнить формулировки назначения программы, состава функций, интерфейсов и состава поставки, а также убедиться, что в документах не используются взаимоисключающие сведения о составе системы. Ожидаемый результат: комплект документации полный, структура документов соответствует требованиям ЕСПД, а сведения в документах согласованы между собой, техническим заданием и актуальными контрактами проекта. Критерий успешности: отсутствуют пропущенные обязательные документы, ошибки в наименованиях и явные расхождения между разделами.

## 6.2. Проверка требований к интерфейсу

Проверка требований к интерфейсу выполняется на веб-приложении, доступном через современный браузер, с учетом ролей пользователя и фактических экранов проекта: авторизация, подтверждение адреса электронной почты, каталог проектов, карточка проекта, форма проекта, очередь модерации, заявки, профиль, административные страницы и интерфейс `django-unfold`. Контроль проводится как визуально, так и через последовательность пользовательских действий, при этом оцениваются доступность элементов управления, логичность навигации, корректность сообщений об ошибках и отсутствие доступа к разделам, не относящимся к роли пользователя.

Исходные данные: учетная запись пользователя одной из поддерживаемых ролей и набор тестовых данных для каталога, заявок и модерации. Действия: выполнить вход, перейти между разделами интерфейса, открыть список проектов и карточку проекта, проверить отображение форм и служебных сообщений, а затем попытаться вызвать недоступные действия для чужой роли. Ожидаемый результат: интерфейс отображает только разрешенные разделы, действия выполняются через понятные элементы управления, ошибки валидации показываются в явном виде, а запрещенные операции завершаются отказом без нарушения состояния страницы. Критерий успешности: навигация единообразна, элементы интерфейса соответствуют роли, пользователь получает однозначную обратную связь по каждому действию.

## 6.3. Проверка требований к функциональным характеристикам

Функциональные характеристики проверяются сценарно, по цепочке от ввода данных до фиксации результата в базе, API и пользовательском интерфейсе. Для каждого сценария используются положительные и отрицательные наборы данных, включая невалидные значения, повторные

операции, попытки доступа без прав и проверку сообщений об ошибках. При испытаниях оцениваются функции каталога проектов, работы с заявками, модерации, импорта XLSX, рекомендаций, избранного, интересов, подтверждения email, домена faculty и публикации доменных событий через outbox API.

**6.3.1. Основной сценарий**

Исходные данные: зарегистрированный пользователь со статусом студента, подтвержденный адрес электронной почты, опубликованные проекты в каталоге и доступная форма подачи заявки. Действия: открыть каталог проектов, применить фильтры и сортировку, перейти в карточку проекта, добавить проект в избранное, задать интересы, подать заявку и затем открыть раздел отслеживания статуса. Ожидаемый результат: каталог отображает релевантные карточки, карточка проекта содержит описание и служебные сведения, выбранный проект сохраняется в избранном, интересы учитываются при формировании рекомендаций, заявка создается и получает корректный статус, а пользователь видит обновление без потери данных формы. Критерий успешности: все операции выполняются без ошибок валидации при корректных данных, а созданные сущности сохраняются и повторно отображаются после обновления страницы.

**6.3.2. Подтверждение email и доступ к учетной записи**

Исходные данные: новый пользователь с адресом электронной почты, не проходившим подтверждение, и корректным паролем. Действия: выполнить регистрацию, получить код подтверждения, ввести код и затем попробовать войти в систему и запросить токен доступа. Ожидаемый результат: до подтверждения email вход и выдача токена ограничены, после успешного подтверждения учетная запись получает доступ к функциям системы, а сообщения об ошибке указывают на причину отказа. Критерий успешности: неподтвержденный пользователь не может получить рабочую сессию, подтвержденный пользователь входит в систему и переходит к разрешенным разделам.

**6.3.3. Импорт и обновление каталога проектов**

Исходные данные: файл XLSX с проектами в поддерживаемом формате или данные, подготовленные для импорта, и учетная запись с правами на выполнение импорта. Действия: загрузить файл через интерфейс или API импорта, запустить обработку, дождаться завершения разбора и открыть каталог проектов после обновления. Ожидаемый результат: корректные строки импорта создают или обновляют проекты, некорректные строки отбрасываются с сообщением об ошибке, а результаты импорта становятся доступны в каталоге и в связанных административных представлениях. Критерий успешности: импорт завершен без падения сервиса, ошибки локализованы по строкам или полям, а валидные проекты отображаются после завершения обработки.

**6.3.4. Поиск и рекомендации**

Исходные данные: текстовый поисковый запрос, набор интересов пользователя и опубликованные проекты, содержащие совпадающие признаки. Действия: выполнить поиск по каталогу, открыть страницу рекомендаций и проверить выдачу проектов на основе интересов пользователя и текстового запроса. Ожидаемый результат: система возвращает релевантные результаты, не смешивает доступные и недоступные для роли объекты и отображает объяснение, если оно предусмотрено

текущей реализацией рекомендаций; при отсутствии семантического ответа допускается корректный переход к резервному ранжированию по ключевым словам без ошибки пользователя. Критерий успешности: результаты поиска и рекомендаций формируются на основе введенных данных, а пустой или нерелевантный ответ корректно обрабатывается интерфейсом и API.

**6.3.5. Подача и обработка заявок**

Исходные данные: студент, опубликованный проект с открытым набором и незавершенная заявка либо отсутствие заявки. Действия: подать заявку на проект, при необходимости отозвать ее, затем выполнить вход под ролью заказчика или сотрудника и обработать заявку через интерфейс рассмотрения. Ожидаемый результат: заявка создается, меняет статус в соответствии с выполненными действиями и отображается у всех участников процесса с актуальным состоянием; при отклонении указывается причина, а при принятии заявка фиксируется как обработанная. Критерий успешности: статус заявки синхронно отражается в интерфейсе и в ответах API, а повторные действия с уже обработанной заявкой не приводят к противоречивому состоянию.

**6.3.6. Создание и модерация проекта**

Исходные данные: учетная запись заказчика или сотрудника с правом создания и модерации проектов, а также шаблон карточки проекта. Действия: создать проект в черновике, заполнить обязательные поля, сохранить изменения, отправить проект на модерацию, затем проверить сценарий возврата на доработку и сценарий публикации. Ожидаемый результат: проект последовательно проходит предусмотренные статусы, история изменений сохраняется, комментарий модератора фиксируется при возврате, а публикация делает проект доступным в каталоге согласно текущим условиям. Критерий успешности: переходы по статусам выполняются только в допустимых направлениях, а попытка выполнить недопустимое действие завершается сообщением об ошибке.

**6.3.7. Экспорт и административные операции**

Исходные данные: учетная запись ЦППРП и набор данных по проектам, заявкам и утвержденным участникам. Действия: открыть административные представления, проверить агрегированные счетчики, просмотреть дедлайны, выполнить обработку доступных административных операций и запросить выгрузку данных в поддерживаемом формате. Ожидаемый результат: административные показатели отображаются корректно, изменения параметров сохраняются, а экспортные данные формируются в соответствии с текущим состоянием системы. Критерий успешности: администратор получает актуальные значения и может завершить регламентные действия без ручного обхода серверных данных.

**6.3.8. Доменные события и синхронизация факультета**

Исходные данные: измененные сущности проекта, заявки, профиля пользователя или объектов домена faculty, а также доступ к outbox API. Действия: выполнить изменение соответствующего объекта, проверить появление события в outbox, при необходимости выполнить подтверждение доставки и запросить актуальное состояние синхронизации по faculty-домену и связям с научными руководителями проектов. Ожидаемый результат: событие фиксируется с корректным типом, внешние процессы получают данные для обновления проекций, а связи между преподавателями,

публикациями, курсами и проектами отражают последнее сохраненное состояние. Критерий успешности: доменные изменения не теряются, outbox-записи доступны для обработки, а актуализация производных представлений выполняется без нарушения целостности данных.

#### 6.3.9. Проверка API по группам маршрутов

Проверка API выполняется, как последовательный маршрут от установления доступа до прикладных и служебных операций. Исходные данные: подтвержденная учетная запись пользователя, доступ к OpenAPI-документации `/api/docs/`, тестовые сущности проектов и заявок, а для служебных сценариев — служебный токен outbox-потребителя. Действия: сначала проверить выдачу токена через `POST /api/v1/auth/token/`, затем последовательно выполнить запросы к `account/me`, `users/me`, `projects`, `applications`, `recs`, `imports`, `faculty` и `outbox`, сравнивая фактические ответы с ожидаемым HTTP-статусом, структурой JSON и ролевыми ограничениями.

Для прикладных ручек успешным результатом считается корректная сериализация данных, соблюдение ограничений по ролям и статусам, а также согласованность результата с интерфейсом и состоянием БД. Для служебных ручек успешным результатом считается наличие обязательных полей контракта, доступность контрольных точек `/api/`, `/api/v1/`, `/api/docs/`, корректная выдача событий `outbox`, обработка подтверждений `ack` и возможность повторного чтения/сопоставления состояния. Критерий успешности: каждая группа маршрутов проходит положительный и отрицательный сценарий, а порядок проверки воспроизводим от выпуска токена до завершения служебного обмена.

#### 6.3.10. Проверка CI/CD и релизного контура

Исходные данные: актуальная ревизия репозитория, настроенный runner или сервер CI, а для production-доставки — серверный стенд с доступом по SSH. Действия: выполнить репозиторные сценарии `./scripts/run-release-gate.sh`, `./scripts/uv-linters.sh`, `./scripts/uv-typecheck.sh`, затем проверить workflow `CI` и `Deploy Production` на предмет корректного запуска стадий, сборки образа и финальной smoke-проверки. При проверке deploy-сценария дополнительно контролируют, что сервер получает актуальную ревизию кода, compose-стек пересобирается или перезапускается без конфликтов, а сервис отвечает через `Nginx`.

Ожидаемый результат: автоматические проверки завершаются без ошибок, релизный образ собирается и публикуется, а после выкладки публичный контур отвечает на базовые запросы и проходит smoke-сценарий. Для практической проверки используются внешние стенды `https://dsa-staging.s3rg.ru/` и `https://dsa.s3rg.ru/`. Критерий успешности: результат локального и серверного release gate совпадает по смыслу, а доставка не требует ручного исправления файлов конфигурации за пределами предусмотренных переменных окружения.

#### 6.3.11. Проверка наблюдаемости

Исходные данные: развернутый observability-контур, включающий `Prometheus` и `Grafana`, а также работающие сервисы `web`, `ml` и `graph`, публикующие метрики. Действия: запустить observability-стек, открыть интерфейс `Prometheus`, убедиться в наличии целей мониторинга, затем открыть `Grafana`, проверить автоматическое подключение источника данных и загрузку панели `dsa-overview`. Если панель мониторинга недоступна напрямую из внешней сети, доступ к ней

организуют через SSH-туннель с локальной переадресацией порта по схеме `ssh -i <key> -l <user> -L 3000:127.0.0.1:3000 <host>`, после чего интерфейс `Grafana` открывают локально через `http://127.0.0.1:3000/`. Конкретные реквизиты подключения в открытую версию документа не включают и хранят во внутреннем эксплуатационном приложении либо в защищенном хранилище секретов. После этого выполняют несколько прикладных запросов к системе и убеждаются, что соответствующие метрики появляются в наблюдаемом контуре.

Ожидаемый результат: мониторинговые цели доступны, панель загружается без ручной настройки, а эксплуатационные показатели серверных сервисов отражают фактическую активность стенда. Критерий успешности: оператор или разработчик может по мониторинговому контуру определить, что сервис запущен, принимает запросы и не находится в состоянии деградации, скрытой от внешнего smoke-контроля.

### 6.3.12. Проверка резервного копирования и восстановления

Исходные данные: запущенный контейнер `postgres`, корректный `env`-файл окружения и каталог для хранения резервных копий. Действия: выполнить `scripts/backup-postgres.sh`, проверить создание каталога запуска, архива дампа, `manifest.txt` и файла контрольной суммы `sha256`, затем на тестовом стенде выполнить восстановление командой `scripts/restore-postgres.sh` для `backup.sql` либо `backup.sql.gz`. После восстановления необходимо повторно открыть прикладные разделы и убедиться, что ключевые данные доступны и согласованы.

Ожидаемый результат: резервная копия формируется без ошибок, содержит служебный манифест и может быть использована для восстановления базы данных. Критерий успешности: процесс восстановления завершается штатно, а данные после восстановления позволяют повторно пройти базовые сценарии входа, просмотра каталога и работы с проектами.

## СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.

# ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

| Лист регистрации изменений | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Номера листов (страниц) | | | | | Всего листов (страниц в докум.) | № документа | Входящий № сопроводительного докум. и дата | Подп. | Дата |
| Изм. | Измененных | Замененных | Новых | Аннулированных | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |